

Optimal Resizable Arrays 论文评述

张艺博 徐黄浩

2026 年 7 月 9 日

摘要

本文评述 Robert E. Tarjan 与 Uri Zwick 的论文 *Optimal resizable arrays*。该论文研究如何在保持数组随机访问效率的同时，降低动态扩缩容带来的额外空间开销。本文将围绕问题模型、相关工作、核心数据结构、复杂度证明以及方法局限展开分析。

目录

1	引言与论文定位	3
2	问题定义与理论模型	4
2.1	可变长数组的抽象接口	4
2.2	内存块模型	4
2.3	两类空间开销	5
2.4	本文的复杂度目标	5
3	文献调研与相关工作	6
3.1	Brodnik 等人的空间下界	6
3.2	Tarjan-Zwick 对下界视角的扩展	6
4	本文核心方法	7
5	正确性证明与理论分析	7
6	技术直觉与方法评价	7
7	总结	7

1 引言与论文定位

数组和可变数组是广泛应用的数据结构，可变数组的内存效率一直是研究者的研究热点。本文所研究的可变数组指的是，可以在末尾添加或者删除元素的数组[1]。为此，很自然想到，如何解决可变长数组的空间分配问题？一个典型场景是：内存已分配的空间已经被填满，此时需要插入新元素，却不能直接在原有内存块末尾追加，因为紧邻的内存可能已被其他对象占用。因此，必须重新申请一块更大的内存，将原有元素全部复制过去，再释放旧块。

这一问题可以从**机制**与**策略**两个层面加以分离与理解。

机制层面

1. 系统提供固定大小的存储块（block）；
2. 存储块可以存放实际数据，也可以存放指向其他块的指针；
3. Grow 操作：申请新块，必要时进行数据迁移；
4. Shrink 操作：删除末尾元素，释放不再需要的空块。

策略层面

1. 块的大小如何选取？
2. 使用多少层指针？Grow 过程中块的大小如何变化？
3. Shrink 的触发条件是什么？何时进行全局重建（rebuild）？

将机制与策略解耦，有助于清晰地分析和设计可变长数组的数据结构。

经典方法：翻倍策略（Doubling） 工业界广泛采用的成熟做法是几何增长策略，其核心规则为：当数组满时，分配一块大小为当前数组两倍的连续内存，并将所有元素复制过去。这一策略的摊还代价为 $O(1)$ ，随机访问同样为 $O(1)$ 最坏情况时间[2]。标准倍增算法可以达到 $O(N)$ （更确切说，应该是 $N + O(N)$ ）空间开销和 $O(1)$ 的时间开销，但其最坏时刻的空间浪费可达 50%。进一步，我们可以将这些策略总结成如下规律(再后面第三节传统文献调研部分有详细说明)：假设每次数组满后，增长因子为 $(1 + \alpha)$ ，那么新数组刚扩展完毕时浪费率为 $\frac{\alpha}{1+\alpha}$ ，但是， α 也不能无限小，当其越小时，时间开销越大，因为平均复制次数增多了，假设数组容量为 C 后触发 grow，新容量变为 $C' = (1 + \alpha)C$ ，每个操作的成本为 $1/\alpha$

本文的目标 能否将空间占用压缩至 $N + o(N)$ ，同时仍然保持 $O(1)$ 的随机访问时间？这正是 Tarjan 和 Zwick 论文试图回答的核心问题[1]。

2 问题定义与理论模型

本节给出本文所讨论的可变长数组（resizable array）的形式化定义，并说明 Tarjan 和 Zwick 论文中用于衡量空间开销的模型[1]。与普通动态序列不同，本文研究的对象更接近栈式数组：元素只能在数组末端增加或删除，但任意位置的读取与修改仍然需要保持常数时间。

2.1 可变长数组的抽象接口

设当前数组为 A ，长度为 N 。论文中的可变长数组支持以下基本操作：

1. **Array()**：创建并返回一个初始为空的数组；
2. **Length()**：返回数组当前长度；
3. **Get(i)**：返回下标为 i 的元素，其中 $0 \leq i < N$ ；
4. **Set(i, a)**：将下标为 i 的元素修改为 a ，其中 $0 \leq i < N$ ；
5. **Grow(a)**：在数组末尾加入一个新元素 a ，使数组长度从 N 变为 $N + 1$ ；
6. **Shrink()**：删除数组末尾元素，使数组长度从 N 变为 $N - 1$ 。

这里最重要的限制是：插入和删除只发生在末端。若允许在任意位置插入或删除，则后续元素的下标都可能发生变化，问题会转化为动态序列维护。该类问题即使不要求极端空间效率，也通常需要更复杂的数据结构和更高的操作复杂度。因此，本文刻意将研究对象限定在末端扩缩容的数组模型上。

2.2 内存块模型

论文假设底层内存管理系统允许申请和释放任意固定长度的数组块。一个可变长数组的具体实现由若干动态分配的块组成，主要包括三类：

1. **数据块（data block）**：存放数组中的实际元素；
2. **索引块（index block）**：存放指向数据块或其他索引块的指针；
3. **辅助块（auxiliary block）**：存放块数量、块长度、层级计数器等辅助信息。

这种模型抽象了实际系统中的动态内存分配行为。若数组必须始终存放在单个连续内存块中，那么扩容时通常只能重新申请更大的块并复制所有元素；而本文允许数组被分散存放在多个块中，再通过索引块维持常数时间访问。这也是后续分层结构能够降低空间浪费的基础。

2.3 两类空间开销

本文的关键建模创新是区分两类空间：

1. **存储与访问空间：** 当数组处于稳定状态、长度为 N 时，为了存储所有元素并支持 `Get` 和 `Set` 所需的空间；
2. **扩缩容临时空间：** 执行 `Grow` 或 `Shrink` 时，在元素移动、块合并或块拆分过程中短暂额外占用的空间。

用论文中的记号表示，若一个实现称为 $(s(N), t(N))$ -implementation，则表示它满足[1]：

$$\text{稳定存储空间} \leq N + s(N),$$

并且在扩缩容操作过程中最多使用：

$$\text{临时峰值空间} \leq N + t(N).$$

这里的 $s(N)$ 衡量平时多出来的空间，例如索引指针、空闲块和辅助信息； $t(N)$ 衡量一次 `grow` 或 `shrink` 过程中可能出现的峰值空间。传统讨论往往把这两种空间混在一起计算，而 Tarjan 和 Zwick 的核心观察是：在很多应用中，平时占用空间比扩缩容瞬间的临时空间更值得优化。例如，一个程序可能同时维护许多可变长数组，但任意时刻只有少数数组正在扩容或缩容。

2.4 本文的复杂度目标

在上述模型下，本文希望同时达到以下目标：

1. `Get` 和 `Set` 操作保持 $O(1)$ 最坏情况时间；
2. `Grow` 和 `Shrink` 操作保持较低的均摊时间；
3. 稳定存储空间尽量接近信息论意义上的下限 N ；
4. 在允许一定临时空间的前提下，突破传统 $N + O(\sqrt{N})$ 空间界限。

论文最终给出的结果是一条由参数 r 控制的折中曲线：对于任意整数 $r \geq 2$ ，可以将稳定存储空间压到 $N + O(N^{1/r})$ ，同时扩缩容时允许短暂使用 $N + O(N^{1-1/r})$ 空间，并保持访问 $O(1)$ 最坏情况时间、扩缩容 $O(r)$ 均摊时间。后文对核心结构和下界证明的分析，都是围绕这组时间-空间折中展开的。

3 文献调研与相关工作

本节梳理可变长数组空间效率问题的前置工作。传统动态数组通过几何扩容获得 $O(1)$ 均摊更新时间，但会保留线性级别的空闲容量[2]；Sitariski 的 hashed array tree 和 Brodnik 等人的结构则把稳定状态下的额外空间降低到 $O(\sqrt{N})$ [3, 4]。Tarjan 和 Zwick 的工作建立在这条脉络上，但其关键贡献不是简单改进已有结构，而是重新区分“稳定存储与访问空间”和“扩缩容临时空间”[1]。

3.1 Brodnik 等人的空间下界

Brodnik 等人的经典结果说明，在不区分临时空间的模型中， $N + \Omega(\sqrt{N})$ 级别的空间占用是不可避免的[4]。这个下界的证明思路很简洁，也解释了为什么后续工作很难在原有模型下突破 $\sqrt{N}$ 壁垒。

考虑只执行 N 次 **Grow** 操作后的状态。设此时数组元素分布在 k 个连续内存块中。由于这些块共同保存 N 个元素，它们的总大小至少为 N 。同时，为了之后能够访问每个块，数据结构至少需要维护到这些块的指针或等价索引信息。

如果 $k \geq \sqrt{N}$ ，那么仅指针数量就已经造成 $\Omega(\sqrt{N})$ 的额外空间。因此总空间至少为

$$N + \Omega(\sqrt{N}).$$

如果 $k < \sqrt{N}$ ，那么在保存 N 个元素的这些块中，必然存在一个大小超过 $\sqrt{N}$ 的块。设这个大块是在数组长度为 $N' \leq N$ 时被分配的。它刚被分配出来时还没有装入原有元素，而旧的 N' 个元素仍然存放在其他地方，所以当时的总空间会超过

$$N' + \sqrt{N'}.$$

因此，无论块数多还是少，某个时刻都会出现 $N + \Omega(\sqrt{N})$ 级别的空间占用。直观地说，块多会带来指针开销，块少则会带来大块分配时的临时空间峰值。

3.2 Tarjan-Zwick 对下界视角的扩展

Tarjan 和 Zwick 并不是否定 Brodnik 等人的下界，而是指出该下界把两类空间混在了一起：一类是平时存储和访问数组需要的空间，另一类是执行 **Grow** 或 **Shrink** 时

短暂出现的临时空间[1]。若用 $(s(N), t(N))$ -implementation 描述二者，则平时总空间为 $N + s(N)$ ，扩缩容过程中的峰值空间为 $N + t(N)$ 。

在这个模型下，Brodnik 下界可以推广为乘积形式。仍然考虑执行 N 次 **Grow** 后的状态。由于每一个数据块都必须可达，块数不能超过额外索引空间的量级，因此可以认为块数至多为 $O(s(N))$ 。这些块一共存放 N 个元素，所以至少有一个块的大小为

$$\Omega\left(\frac{N}{s(N)}\right).$$

当这个大块刚被分配出来时，它尚未存放旧元素，旧元素仍然需要由原有块保存。因此在那次 **Grow** 的过程中，临时额外空间至少达到该大块的大小量级，即

$$t(N) = \Omega\left(\frac{N}{s(N)}\right).$$

于是得到

$$s(N)t(N) = \Omega(N).$$

忽略常数后，这就是论文中 $s(N)t(N) \geq N$ 的含义。它表明平时额外空间和扩缩容临时空间不能同时任意小。若希望稳定存储空间为 $N + O(N^{1/r})$ ，则临时空间至少需要达到 $N + \Omega(N^{1-1/r})$ 。Tarjan 和 Zwick 后续给出的构造正好达到这一量级，因此它在两类空间的 trade-off 上是最优的。

4 本文核心方法

5 正确性证明与理论分析

6 技术直觉与方法评价

7 总结

参考文献

- [1] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays, 2023.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Edward Sitariski. Algorithm alley: Hats: Hashed array trees. *Dr. Dobb's Journal of Software Tools*, 21(9):107, 1996.

- [4] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Proceedings of the 6th International Workshop on Algorithms and Data Structures (WADS)*, pages 37–48, 1999.